# Verilog 单周期 CPU 设计文档

## 一、CPU 设计方案综述

### 1.1 概述

1. 基于Logisim单周期CPU，进行Verilog语言描述。
2. 支持指令集包括add、sub、ori、lui、beq、lw、sw、nop、jr、jal。
3. add和sub不考虑溢出。

### 1.2 模块端口及功能定义

#### 1.2.1 IFU（取指令单元）

- **介绍**：内部包括 PC（程序计数器）、IM（指令存储器）及相关逻辑，PC 用寄存器实现，具有异步复位功能，复位值为起始地址，起始地址为0x00003000，地址范围为0x00003000 ~ 0x00006FFF。IM 用 ROM 实现，容量为 4096 × 32bit，ROM 内部的起始地址是从 0 开始的，即 ROM 的 0 位置存储的是 PC 为 0x00003000 的指令，每条指令是一个 32bit 常数。
- **端口定义**：

| 端口 | 方向 | 位宽 | 功能描述 |
|---|---|---|---|
| Clk | I | 1 | 时钟信号 |
| Reset | I | 1 | 复位信号 |
| NPC | I | 32 | 下一条指令的地址 |
| PC | O | 32 | 当前指令的地址 |
| instr | O | 32 | 被取出的指令 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
|---|---|---|
| 1 | 复位 | 若 Reset 有效，PC 重置为 `0x00003000` |
| 2 | 取指令 | PC 从 IFU 中取出对应指令 |
| 3 | 跳转 | 当 Branch 有效时，执行 `PC = PC + 4 + imm` 操作 |

#### 1.2.2 IM （指令存储器）

- **介绍**：存储所有的指令，容量为 4096 × 32bit。
- **端口定义**：

| 端口 | 方向 | 位宽 | 描述 |
|---|---|---|---|
| addr | I | 32 | 指令的地址 |
| instr | O | 32 | 指令的内容 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
|---|---|---|
| 1 | 读数据 | 读出 addr 地址对应指令至 instr |

#### 1.2.3 NPC （指令地址计算器）

- **介绍**：根据当前指令地址和输入的立即数，在NPCsrc指导下计算下一条指令地址。
- **端口定义**：

| 端口 | 方向 | 位宽 | 描述 |
|---|---|---|---|
| NPCsrc | I | 1 | NPC 计算选择信号 |
| Immediate | I | 32 | 立即数 |
| PC | I | 32 | 当前指令的地址 |
| NPC | O | 32 | 下一条指令的地址 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
|---|---|---|
| 1 | 计算下一条指令地址 | 根据当前指令地址和输入的立即数，在NPCsrc指导下计算下一条指令地址 |

### 1.2.4 GRF（通用寄存器组）

- **介绍**：用具有写使能的寄存器实现，寄存器总数为 32 个，具有异步复位功能，其中0 号寄存器的值始终保持为 0，其他寄存器初始值（复位后）均为 0。
- **端口定义**：

| 端口 | 方向 | 位宽 | 描述 |
|---|---|---|---|
| Clk | I | 1 | 时钟信号 |
| Reset | I | 1 | 复位信号 |
| En | I | 1 | 写使能信号 |
| A1 | I | 5 | 第一个寄存器读入地址 |
| A2 | I | 5 | 第二个寄存器读入地址 |
| A3 | I | 5 | 寄存器写地址 |
| WD | I | 32 | 写入的数据 |
| RD1 | O | 32 | A1 所指寄存器输出数据 |
| RD2 | O | 32 | A2 所指寄存器输出数据 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
|---|---|---|
| 1 | 复位 | Reset 有效时，所有寄存器清零 |
| 2 | 读数据 | 读出 A1 或 A2 所指寄存器数据至 RD1 或 RD2 |
| 3 | 写数据 | 将 WD 写入 A3 所指寄存器 |

### 1.2.5 ALU（算术逻辑单元）

- **介绍**：提供 32 位加、减、或运算及大小比较功能，加减法按无符号处理（不考虑溢出）。
- **端口定义**：

| 端口 | 方向 | 位宽 | 描述 |
|---|---|---|---|
| A | I | 32 | 第一个操作数 |
| B | I | 32 | 第二个操作数 |
| Op | I | 4 | 选择进行什么运算 |
| C | O | 32 | 运算结果 |
| beZero | O | 1 | 展示 C 是否等于0 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
| --- | --- | --- |
| 1 | 无符号加法运算 | Op=4'b0000 时，执行加法运算，不考虑溢出 |
| 2 | 无符号减法运算 | Op=4'b0001 时，执行减法运算，不考虑溢出 |
| 3 | 按位或运算 | Op=4'd0010 时，执行按位或运算 |
| 4 | 小于判断 | Op=4'd0011 时，判断第一个操作数是否小于第二个 |
| 5 | 加载至高位运算 | Op=4'd0100 时，把第二个操作数逻辑左移16个二进制位 |
| 6 | 判断结果是否为0 | beZero=1 时，表示 C=0 |

### 1.2.6 DM（数据存储器）

- **介绍**：使用 RAM 实现，容量为 3072 × 32bit，具有异步复位功能，复位值为 0x00000000，起始地址为0x00000000，地址范围为0x00000000 ~ 0x00002FFF。RAM 应使用双端口模式，即设置 RAM 的 Data Interface 属性为 Separate load and store ports。
- **端口定义**：

| 端口 | 方向 | 位宽 | 描述 |
| --- | --- | --- | --- |
| Clk | I | 1 | 时钟信号 |
| Reset | I | 1 | 复位信号 |
| WE | I | 1 | 写使能信号 |
| Addr | I | 5 | 读写地址 |
| WD | I | 32 | 写入的数据 |
| RD | O | 32 | 读出的数据 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
| --- | --- | --- |
| 1 | 复位 | reset=1 时，DM 数据清零 |
| 2 | 写数据 | WE=1 时，将 WD 写入 Addr 所指的存储空间 |
| 3 | 读数据 | WE=0 时，RD 将读出 Addr 所指的存储空间的数据 |

### 1.2.7 EXT（扩展单元）

- **介绍**：实现16位立即数到32位立即数的零扩展和符号扩展。
- **端口定义**：

| 端口 | 方向 | 位宽 | 描述 |
| --- | --- | --- | --- |
| Immediate | I | 16 | 待扩展的立即数 |
| mode | I | 1 | 控制如何扩展立即数 |
| Result | O | 32 | 扩展后的立即数 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
| --- | --- | --- |
| 1 | 零扩展 | mode=0 时，进行零扩展 |
| 2 | 符号扩展 | mode=1 时，进行符号扩展 |

### 1.2.8 CTRL（控制器）

- **介绍**：使用和逻辑和或逻辑，把操作码和功能码翻译成控制信号。
- **端口定义**：

| 端口 | 方向 | 位宽 | 描述 |
|---|---|---|---|
| Opcode | I | 6 | 操作码 |
| Funct | I | 6 | 功能码 |
| RegDst | O | 1 | 寄存器写地址选择信号 |
| ALUSrc | O | 1 | ALU第二个操作数选择信号 |
| Mem2Reg | O | 1 | 寄存器写入数据选择信号 |
| RegWrite | O | 1 | 寄存器写使能信号 |
| MemWrite | O | 1 | 数据存储器写使能信号 |
| Branch | O | 1 | 分支信号 |
| ExtOp | O | 1 | 立即数扩展操作选择信号 |
| ALUop | O | 4 | ALU 操作选择信号 |

- **功能定义**：

| 序号 | 功能名称 | 功能描述 |
|---|---|---|
| 1 | 生成控制信号 | 根据输入的操作码和功能码，生成控制信号 |

## 1.3 控制器控制机制

### 控制信号产生

通过与或门阵列生成控制信号，控制信号真值表如下：

| Instr | Opcode | Funct | RegDst | ALUSrc | Mem2Reg | RegWrite | MemWrite | Branch | ExtOp | ALUop | Jump | Jal | JR |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ADD | 000000 | 100000 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 4'b0000 | 0 | 0 | 0 |
| SUB | 000000 | 100010 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 4'b0001 | 0 | 0 | 0 |
| ORI | 001101 | 000000 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 4'b0010 | 0 | 0 | 0 |
| LUI | 001111 | 000000 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 4'b0100 | 0 | 0 | 0 |
| BEQ | 000100 | 000000 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 4'b0001 | 0 | 0 | 0 |
| LW | 100011 | 000000 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 4'b0000 | 0 | 0 | 0 |
| SW | 101011 | 000000 | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 4'b0000 | 0 | 0 | 0 |
| NOP | 000000 | 000000 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4'b0000 | 0 | 0 | 0 |
| JR | 000000 | 001000 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4'b0000 | 0 | 0 | 1 |
| JAL | 000011 | 000000 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4'b0000 | 1 | 1 | 0 |

# 二、测试方案

### 测试 `ori` 指令

```
34011234 3402ffff 34234321 34040000
```

### 测试 lui 指令

```
3c051234 3c06ffff 3c070000 3c089a7c
```

### 测试 add 指令

```
00221820 00212020 00c72820 00443020
```

### 测试 sub 指令

```
00221822 00412022 00212822 00663822
```

### 测试 lw 指令

```
8c020004 8c040008 8c050000 8c060014
```

### 测试 sw 指令

```
ac010004  ac02000c  ac030010  ac040018
```

### 测试 beq 指令

```
10220002 10250001 10c00001 10890003
```

### 测试 nop 指令

```
00000000 00000000 00000000 00000000
```

### 测试 jr 指令

```
03E00008 00800008 02200008 03400008
```

### 测试 jal 指令

```
0C000008 0C00001C 0C000030 0C000044
```

### 综合测试

```
341c0000 341d0000 34013456 00210820
ac010004 3c027878 00411822 3c051234
34040005 00000000 ac85ffff 8c83ffff
10650003 00000000 10000011 00000000
34670404 10e3000e 00000000 3c087777
3508ffff 00080022 34001100 00e65020
34080000 34090001 340a0001 010a4020
1109fffe 0c000c22 00000000 014a5020
1000ffff 014a5020 03e00008 00000000
```

## 三、思考题

**Q**：阅读下面给出的 DM 的输入示例中（示例 DM 容量为 4KB，即 32bit × 1024字），根据你的理解回答，这个 addr 信号又是从哪里来的？地址信号 addr 位数为什么是 [11:2] 而不是 [9:0]？
**A**：每4个byte存储一个32位数。DM的addr输入采用的是byte，但RAM寻址时是按照word来寻址，所以addr需要右移2位。

**Q**：思考上述两种控制器设计的译码方式，给出代码示例，并尝试对比各方式的优劣。
**A**：给出示例如下

指令对应的控制信号如何取值：

```
if(Opcode6'b000000 && Funct6'b100000)
{RegDst=1'b1,ALUSrc=1'b0,Mem2Reg=1'b0,RegWrite=1'b1,MemWrite=1'b0,Branch=1'b0,ExtOp=1'b1,ALUop=4'b0000,Jump=1'b0,Jal=1'b0,JR=1'b0};
……
```

控制信号每种取值所对应的指令:

```
assign RegWrite = add || sub || ori || lui || lw || jal;
assign ExtOp = add || sub || beq || lui || lw || sw;
……
```

各自的优劣：第一种易于阅读和直接理解代码逻辑，但是书写起来比较繁琐；第二种书写起来比较简单，但是不易理解代码逻辑。

**Q**：在相应的部件中，复位信号的设计都是同步复位，这与 P3 中的设计要求不同。请对比同步复位与异步复位这两种方式的 reset 信号与 clk 信号优先级的关系。

**A**：编写Verilog时，对于同步复位clk比reset高一个优先级，对于异步复位clk和reset优先级相同。

**Q**：C 语言是一种弱类型程序设计语言。C 语言中不对计算结果溢出进行处理，这意味着 C 语言要求程序员必须很清楚计算结果是否会导致溢出。因此，如果仅仅支持 C 语言，MIPS 指令的所有计算指令均可以忽略溢出。 请说明为什么在忽略溢出的前提下，addi 与 addiu 是等价的，add 与 addu 是等价的。提示：阅读《MIPS32® Architecture For Programmers Volume II: The MIPS32® Instruction Set》中相关指令的 Operation 部分。

**A**：addu的计算逻辑是最简单的逢二进一。add在不考虑溢出的情况下，计算逻辑是即使最高位的计算发生了溢出也不报错，所以和addu的逻辑完全一致。addi和addiu同理，只是一个操作数换成了立即数。